

Week 3 Lab

Your First Java Programs

DELIVERED BY: Teaching Assistants

FOLLOWS: Lecture — From Flowchart to Java Code

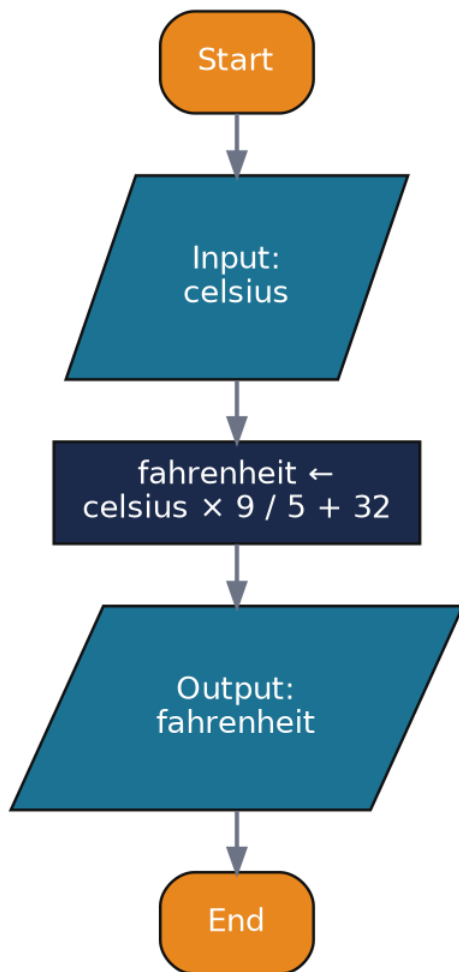
FORMAT: Type, compile and run each program; every solution below was compiled and run

GROUND RULES

- Part A–C turn flowcharts you already know into Java. Part D is about type conversion. Part E combines everything.
- Each page shows the flowchart (left), a complete Java solution (right) and a real sample run. Cover the solution while you work.
- File name = class name: Q7 lives in Q7.java. Compile with `javac Q7.java`, run with `java Q7` — or just `java Q7.java`.
- Every program that reads input needs: `import java.util.Scanner;` and `Scanner in = new Scanner(System.in);`

Q1. Translate the flowchart into Java: read a temperature in Celsius and print it in Fahrenheit with one decimal.

FLOWCHART



JAVA SOLUTION

Q1.java

```
import java.util.Scanner;

public class Q1 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        double celsius = in.nextDouble();
        double fahrenheit = celsius * 9 / 5 + 32;
        System.out.printf("%.1f%n", fahrenheit);
    }
}
```

SAMPLE RUN

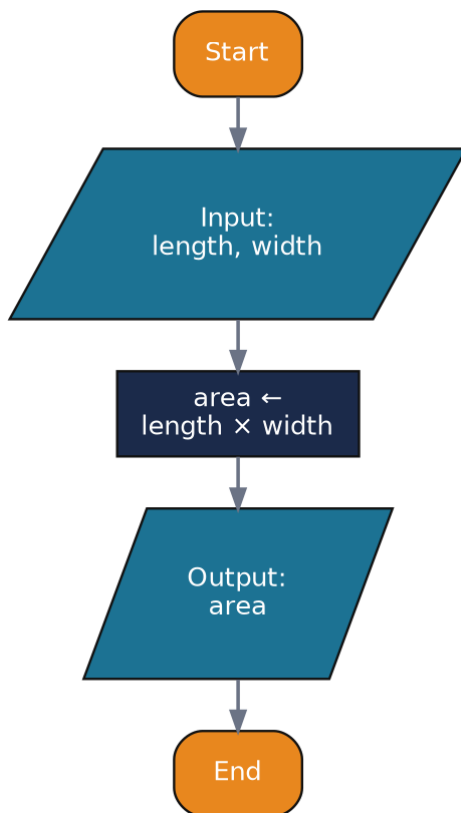
```
$ java Q1
36.6
97.9
```

HINT

Write $\text{celsius} * 9 / 5$, not $9 / 5 * \text{celsius}$ — the second form computes $9 / 5 = 1$ first (integer division).

Q2. Read the length and width of a rectangle (whole numbers) and print its area.

FLOWCHART



JAVA SOLUTION

Q2.java

```
import java.util.Scanner;

public class Q2 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int length = in.nextInt();
        int width = in.nextInt();
        int area = length * width;
        System.out.println(area);
    }
}
```

SAMPLE RUN

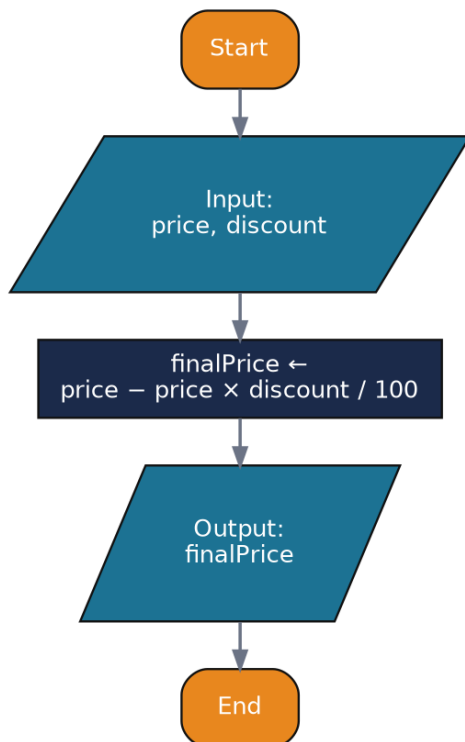
```
$ java Q2
5 8
40
```

HINT

Both inputs are whole numbers, so use `int` and `nextInt()` — no decimals needed.

Q3. Read a price and a discount percentage; print the final price with two decimals.

FLOWCHART



JAVA SOLUTION

Q3.java

```
import java.util.Scanner;

public class Q3 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        double price = in.nextDouble();
        double discount = in.nextDouble();
        double finalPrice = price - price * discount / 100;
        System.out.printf("%.2f\n", finalPrice);
    }
}
```

SAMPLE RUN

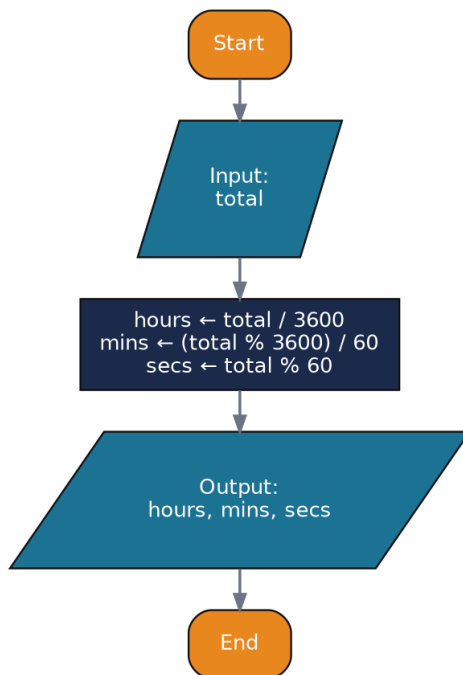
```
$ java Q3
250 20
200.00
```

HINT

$\text{price} * \text{discount} / 100$ is the discount amount. Because discount is a double, the division keeps its fraction.

Q4. Read a number of seconds; print it as hours, minutes and seconds.

FLOWCHART



JAVA SOLUTION

Q4.java

```
import java.util.Scanner;

public class Q4 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int total = in.nextInt();
        int hours = total / 3600;
        int mins = (total % 3600) / 60;
        int secs = total % 60;
        System.out.printf("%d h %d m %d s\n", hours, mins, secs);
    }
}
```

SAMPLE RUN

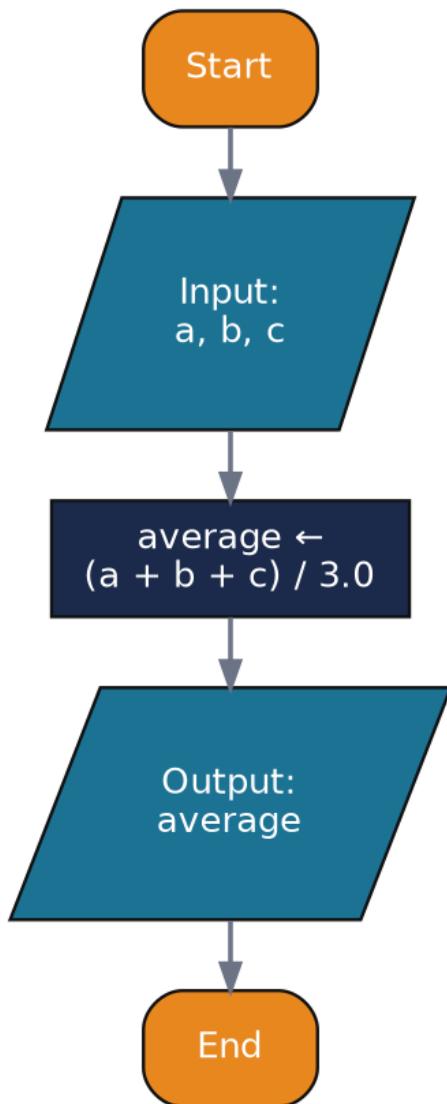
```
$ java Q4
3725
1 h 2 m 5 s
```

HINT

total / 3600 gives whole hours; total % 3600 is what is left; then / 60 and % 60 split minutes from seconds.

Q5. Read three integers and print their average with two decimals.

FLOWCHART



JAVA SOLUTION

Q5.java

```
import java.util.Scanner;

public class Q5 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int a = in.nextInt();
        int b = in.nextInt();
        int c = in.nextInt();
        double average = (a + b + c) / 3.0;
        System.out.printf("%.2f\n", average);
    }
}
```

SAMPLE RUN

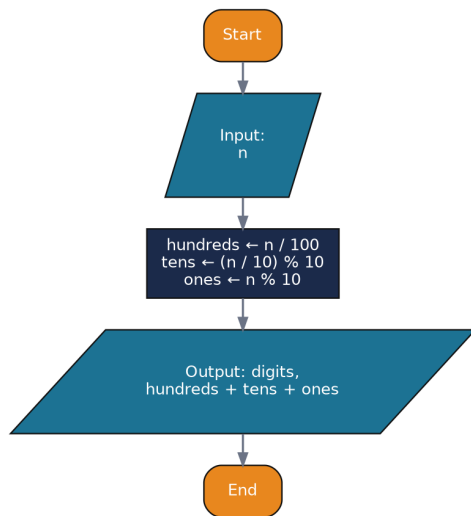
```
$ java Q5
7 8 10
8.33
```

HINT

Dividing by 3 would give 8.00 (int division drops .33). Dividing by 3.0 keeps the fraction: 8.33.

Q6. Read a three-digit number; print its digits separated by spaces, then the sum of the digits.

FLOWCHART



JAVA SOLUTION

Q6.java

```
import java.util.Scanner;

public class Q6 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int n = in.nextInt();
        int hundreds = n / 100;
        int tens = (n / 10) % 10;
        int ones = n % 10;
        System.out.println(hundreds + " " + tens + " " + ones);
        System.out.println("Sum: " + (hundreds + tens + ones));
    }
}
```

SAMPLE RUN

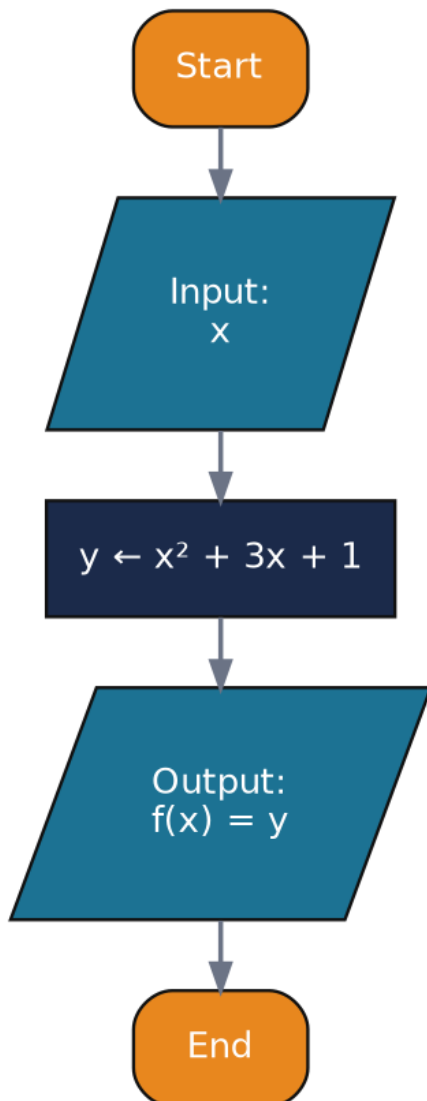
```
$ java Q6
472
4 7 2
Sum: 13
```

HINT

$n / 100$ is the hundreds digit, $(n / 10) \% 10$ the tens, $n \% 10$ the ones. In the string, put the arithmetic in parentheses.

Q7. Read x and print $f(x) = x^2 + 3x + 1$ (the read x , compute $f(x)$, print pattern).

FLOWCHART



JAVA SOLUTION

Q7.java

```
import java.util.Scanner;

public class Q7 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        double x = in.nextDouble();
        double y = Math.pow(x, 2) + 3 * x + 1;
        System.out.println("f(" + x + ") = " + y);
    }
}
```

SAMPLE RUN

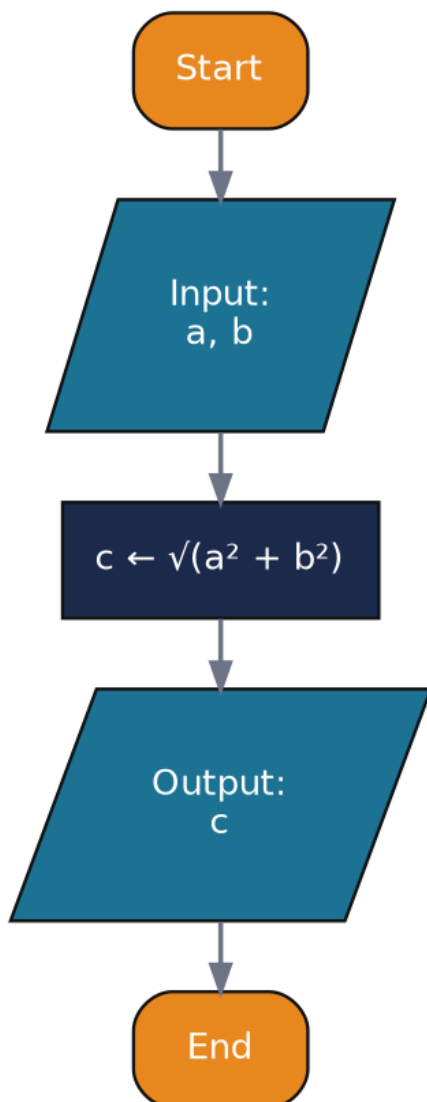
```
$ java Q7
2.5
f(2.5) = 14.75
```

HINT

`Math.pow(x, 2)` and `x * x` give the same result. `Math.pow` returns a double.

Q8. Read the two legs of a right triangle; print the hypotenuse with two decimals.

FLOWCHART



JAVA SOLUTION

Q8.java

```
import java.util.Scanner;

public class Q8 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        double a = in.nextDouble();
        double b = in.nextDouble();
        double c = Math.sqrt(a * a + b * b);
        System.out.printf("Hypotenuse: %.2f\n", c);
    }
}
```

SAMPLE RUN

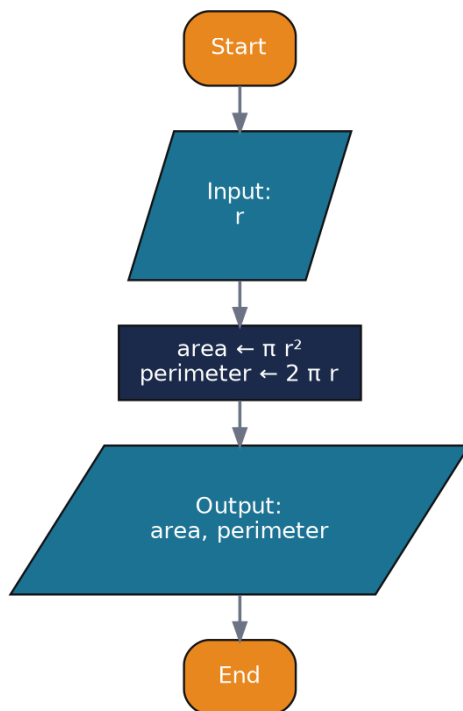
```
$ java Q8
3 4
Hypotenuse: 5.00
```

HINT

`c = Math.sqrt(a * a + b * b)`. Use `%.2f` in `printf` to round only the display.

Q9. Read a radius; print the circle's area and perimeter with two decimals.

FLOWCHART



JAVA SOLUTION

Q9.java

```
import java.util.Scanner;

public class Q9 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        double r = in.nextDouble();
        double area = Math.PI * Math.pow(r, 2);
        double perimeter = 2 * Math.PI * r;
        System.out.printf("Area: %.2f\n", area);
        System.out.printf("Perimeter: %.2f\n", perimeter);
    }
}
```

SAMPLE RUN

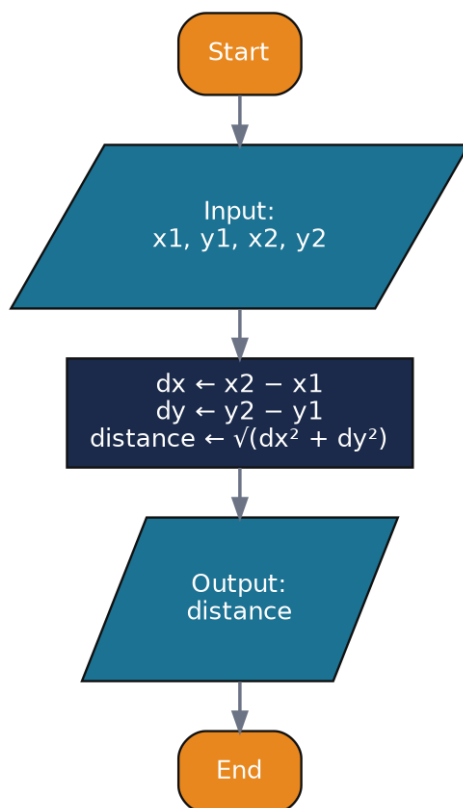
```
$ java Q9
2.5
Area: 19.63
Perimeter: 15.71
```

HINT

Math.PI is a ready-made constant. Use %n at the end of each printf line.

Q10. Read the coordinates of two points; print the distance between them with two decimals.

FLOWCHART



JAVA SOLUTION

Q10.java

```
import java.util.Scanner;

public class Q10 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        double x1 = in.nextDouble();
        double y1 = in.nextDouble();
        double x2 = in.nextDouble();
        double y2 = in.nextDouble();
        double dx = x2 - x1;
        double dy = y2 - y1;
        double distance = Math.sqrt(dx * dx + dy * dy);
        System.out.printf("Distance: %.2f\n", distance);
    }
}
```

SAMPLE RUN

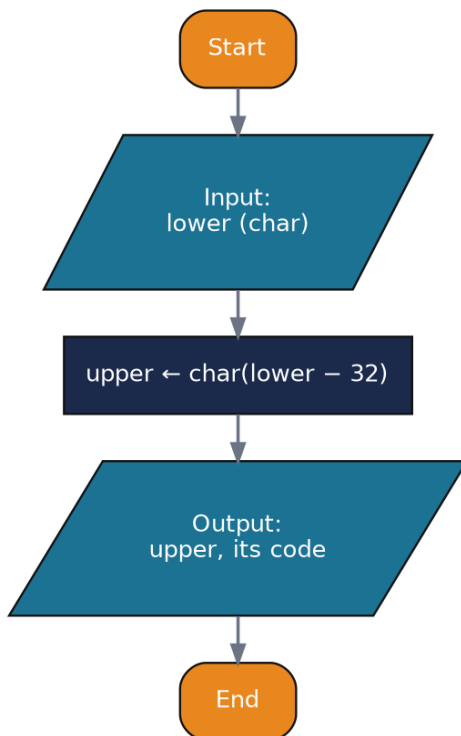
```
$ java Q10
1 2 4 6
Distance: 5.00
```

HINT

Subtract first (dx, dy), then take Math.sqrt of the sum of the squares. Four values to read: x1, y1, x2, y2.

Q11. Read one lowercase letter; print the matching uppercase letter and its character code.

FLOWCHART



JAVA SOLUTION

Q11.java

```
import java.util.Scanner;

public class Q11 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        char lower = in.next().charAt(0);
        char upper = (char) (lower - 32);
        System.out.println(upper);
        System.out.println("Code: " + (int) upper);
    }
}
```

SAMPLE RUN

```
$ java Q11
c
C
Code: 67
```

HINT

In ASCII, 'a' - 'A' is 32. Subtracting 32 gives the uppercase letter as an int; the cast (char) turns it back into a character. next().charAt(0) reads one char.

Q12. Predict the output of the program below line by line, then run it to check yourself.

PROGRAM

```
Q12.java
public class Q12 {
    public static void main(String[] args) {
        int a = 9, b = 4;
        double d = 2.5;
        System.out.println(a / b);
        System.out.println(a % b);
        System.out.println((double) a / b);
        System.out.println((int) d + a);
        System.out.println("Sum: " + a + b);
        System.out.println("Sum: " + (a + b));
        System.out.println('A' + 1);
        System.out.println((char) ('A' + 1));
    }
}
```

ACTUAL OUTPUT (check against your prediction)

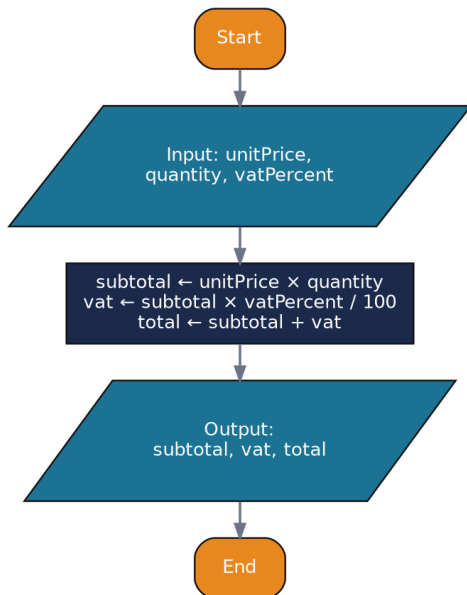
```
$ java Q12
2
1
2.25
11
Sum: 94
Sum: 13
66
B
```

WHY

expression	prints	why
a / b	2	int / int drops the fraction (9 / 4 is 2.25, so 2 is kept)
a % b	1	remainder of 9 ÷ 4
(double) a / b	2.25	cast first, then divide: 9.0 / 4
(int) d + a	11	(int) 2.5 is 2, plus 9
"Sum: " + a + b	Sum: 94	left to right: "Sum: 9" then "Sum: 94"
"Sum: " + (a + b)	Sum: 13	parentheses force the addition first
'A' + 1	66	a char in arithmetic becomes its int code
(char) ('A' + 1)	B	cast the int 66 back to a character

Q13. Receipt: read a unit price, a quantity and a VAT percentage; print the subtotal, the VAT and the total, right-aligned with two decimals.

FLOWCHART



JAVA SOLUTION

Q13.java

```
import java.util.Scanner;

public class Q13 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        double unitPrice = in.nextDouble();
        int quantity = in.nextInt();
        int vatPercent = in.nextInt();
        double subtotal = unitPrice * quantity;
        double vat = subtotal * vatPercent / 100;
        double total = subtotal + vat;
        System.out.printf("Subtotal: %8.2f\n", subtotal);
        System.out.printf("VAT (%d%%): %6.2f\n", vatPercent, vat);
        System.out.printf("Total: %8.2f\n", total);
    }
}
```

SAMPLE RUN

```
$ java Q13
12.5 3 20
Subtotal:    37.50
VAT (20%):   7.50
Total:      45.00
```

HINT

%8.2f prints every number 8 characters wide so the decimal points line up. %% prints a literal percent sign.

WRAP-UP & INSTRUCTOR NOTES

Wrap-up check (5 min)

Before leaving, each student should be able to answer out loud, pointing at their own code:

- “Why does $(a + b + c) / 3$ lose the decimals, and what are two ways to fix it?”
- “Why does `"Sum: " + a + b` print 94, and how do I get 13?”

Instructor notes

- Parts A–C are the Week 1–2 flowcharts in Java. Ask students to point at the flowchart symbol each Java line comes from: Input → `nextX()`, Process → assignment, Output → `println` / `printf`.
- Most common first-day errors: forgetting `import java.util.Scanner`; a file name that doesn't match the class name; a missing semicolon; and writing `int` where a `double` is needed. Have students read the compiler message top-down — file, line, caret.
- Q1 and Q5 are deliberate integer-division traps. Let students hit the wrong answer first (try `9 / 5 * celsius`, and `/ 3` instead of `/ 3.0`), then explain.
- Q11 and Q12 target type conversion. Q12 is a paper exercise: insist on a written prediction before anyone runs the program.
- Q13 is the capstone: three inputs, three computed values, aligned output. Expect most students to need the full session for it; formatting with `%8.2f` is the new idea.
- The `nextInt()` / `nextLine()` trap from the lecture does not appear in these programs. If a student combines the two on their own, use it as a teachable moment.
- Next lecture: `if` / `else` and boolean logic — the decision diamonds from Weeks 1–2, in Java.